

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №5 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Применение класса Thread»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 7

Вопросы для защиты работы **10**

Выводы **11**

Постановка задачи

Целью данной лабораторной работы является изучение возможностей встроенного класса `Thread`

Задачи лабораторной работы:

- Научиться создавать потоки
- Научиться использовать массивы потоков
- Научиться осуществлять мониторинг потоков

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n thread_101
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

В соответствии с примером реализуем функцию `Main`, в которой определим массив потоков, константу, равную числу создаваемых потоков, цикл, запускающий поочередно каждый поток, и ожидание ввода для того, чтобы дождаться выполнения вычислений:

```
static void Main(string[] args)
{
        const int NUMBER_OF_THREADS = 3;
        Thread[] threads = new Thread[NUMBER_OF_THREADS];
        for (int i = 0; i < NUMBER_OF_THREADS; i++)
        {
                threads[i] = new Thread(CalcStatistics);
                threads[i].Start();
        }
        Console.ReadKey();
}
```

По индивидуальному заданию необходимо написать код, делающий подсчет количества чисел в случайно сгенерированной матрице. Воспользуемся алгоритмом:

1. Определим константы, задающие максимальную величину числа в матрице и величину квадратной матрицы

2. Объявим вложенный список, который определяет матрицу

3. Заполним матрицу значениями

4. Создадим словарь с ключом типа `String` (подсчитываемый элемент) и значением типа `int` (количество элемента).

5. Пройдемся по элементам матрицы и, если элемент словаря не создан - создаем его со значением 0. Инкрементируем значение по текущему ключу

Код, реализующий данный алгоритм, приведен в листинге ниже:

```
static void CalcStatistics()
{
        int threadId = Thread.CurrentThread.ManagedThreadId;
        Console.WriteLine("Поток № {0}, Инициализация переменных...", threadId);
        const int MATRIX_SIZE = 1_000;
        const int MAX_NUMBER = 20;
        Random random = new Random();
        List<List<int>> matrix = new List<List<int>>();
        Console.WriteLine("Поток № {0}, Создание матрицы...", threadId);
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
                matrix.Add(new List<int>(MATRIX_SIZE));
        }
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
                for (int j = 0; j < MATRIX_SIZE; j++)
                {
                        matrix[i].Add(random.Next(MAX_NUMBER));
                }
        }

        Dictionary<String, int> statistics = new Dictionary<string, int>();
        Console.WriteLine("Поток № {0}, Подсчет статистики...", threadId);

        for (int i = 0; i < MATRIX_SIZE; i++)
        {
                for(int j = 0; j < MATRIX_SIZE; j++)
                {
                        string key = matrix[i][j].ToString();
                        if (!statistics.ContainsKey(key))
                        {
                                statistics.Add(key, 0);
                        }
                        statistics[key]++;
                }
        }
        Console.WriteLine("Поток № {0}, Завершение процесса. Результаты:", threadId);
        foreach (KeyValuePair<string, int> item in statistics)
        {
                Console.WriteLine(item.ToString());
        }
}
```

Важно, что дополнительно для идентификации вывода потоков определяется переменная `threadId`, которая добавляется к выводу по результату ключевых действий кода.

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace thread_101
{
        internal class Program
        {
                static void Main(string[] args)
                {
                        const int NUMBER_OF_THREADS = 3;
                        Thread[] threads = new Thread[NUMBER_OF_THREADS];
                        for (int i = 0; i < NUMBER_OF_THREADS; i++)
                        {
                                threads[i] = new Thread(CalcStatistics);
                                threads[i].Start();
                        }
                        Console.ReadKey();
                }
                static void CalcStatistics()
                {
                        int threadId = Thread.CurrentThread.ManagedThreadId;
                        Console.WriteLine("Поток № {0}, Инициализация переменных...",
                            threadId);
                        const int MATRIX_SIZE = 1_000;
                        const int MAX_NUMBER = 20;
                        Random random = new Random();
                        List<List<int>> matrix = new List<List<int>>();
                        Console.WriteLine("Поток № {0}, Создание матрицы...", threadId)
                            ;
                        for (int i = 0; i < MATRIX_SIZE; i++)
                        {
                                matrix.Add(new List<int>(MATRIX_SIZE));
                        }
                        for (int i = 0; i < MATRIX_SIZE; i++)
                        {
                                for (int j = 0; j < MATRIX_SIZE; j++)
                                {
                                        matrix[i].Add(random.Next(MAX_NUMBER));
                                }
                        }

                        Dictionary<String, int> statistics = new Dictionary<string, int
                            >();
                        Console.WriteLine("Поток № {0}, Подсчет статистики...",
                            threadId);

```

```
47                      for (int i = 0; i < MATRIX_SIZE; i++)
48                      {
49                              for(int j = 0; j < MATRIX_SIZE; j++)
50                              {
51                                      string key = matrix[i][j].ToString();
52                                      if (!statistics.ContainsKey(key))
53                                      {
54                                              statistics.Add(key, 0);
55                                      }
56                                      statistics[key]++;
57                              }
58                      }
59                      Console.WriteLine("Поток № {0}, Завершение процесса.
                            Результаты:", threadId);
60                      foreach (KeyValuePair<string, int> item in statistics)
61                      {
62                              Console.WriteLine(item.ToString());
63                      }
64              }
65      }
66  }
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
1  dotnet build; dotnet run
```

Вывод программы:

```
Поток № 5, Инициализация переменных...
Поток № 4, Инициализация переменных...
Поток № 4, Создание матрицы...
Поток № 5, Создание матрицы...
Поток № 3, Инициализация переменных...
Поток № 3, Создание матрицы...
Поток № 4, Подсчет статистики...
Поток № 3, Подсчет статистики...
Поток № 5, Подсчет статистики...
Поток № 4, Завершение процесса. Результаты:
[10, 50077]
[8, 50313]
[1, 50270]
[6, 50447]
[11, 50139]
[7, 50016]
```

```
[19, 50045]
[16, 49984]
[15, 50165]
[13, 50230]
[4, 49490]
[18, 49710]
[17, 50190]
[12, 49996]
[3, 49597]
[0, 49810]
[5, 49741]
[9, 49488]
[14, 49992]
[2, 50300]
Поток № 3, Завершение процесса. Результаты:
[10, 50077]
[8, 50313]
[1, 50270]
[6, 50447]
[11, 50139]
[7, 50016]
[19, 50045]
[16, 49984]
[15, 50165]
[13, 50230]
[4, 49490]
[18, 49710]
[17, 50190]
[12, 49996]
[3, 49597]
[0, 49810]
[5, 49741]
[9, 49488]
[14, 49992]
[2, 50300]
Поток № 5, Завершение процесса. Результаты:
[10, 50077]
[8, 50313]
[1, 50270]
[6, 50447]
[11, 50139]
[7, 50016]
[19, 50045]
```

```
[16, 49984]
[15, 50165]
[13, 50230]
[4, 49490]
[18, 49710]
[17, 50190]
[12, 49996]
[3, 49597]
[0, 49810]
[5, 49741]
[9, 49488]
[14, 49992]
[2, 50300]
```

Программа выполняется корректно. Вывод содержит информацию о работе каждого потока. Вывод находится не в порядке вызова потоков, так как регулируемый ОС пул потоков сам определяет очередность их работы.

Вопросы для защиты работы

1. **В каком пространстве имен определен класс `Thread`? Поясните назначение класса `Thread`.**

   `Thread` Определен в пространстве имен `System.Threading`. Он нужен для создания потоков и управления параллельными задачами, работающих в них.

2. **Как получить идентификатор текущего потока?**

   Идентификатор текущего потока можно получить через поле `Thread.CurrentThread.ManagedThreadId`.

3. **Какой метод осуществляет запуск метода на выполнение в потоке?** Запуск метода в потоке достигается при помощи вызова `Start()`.

4. **Для чего необходимы делегаты при использовании класса Thread?**

   - Чтобы уметь запускать в потоке лямбда-выражения
   - Чтобы унифицировать сигнатуру передаваемой в поток функции

5. **Как организовать несколько потоков в программе?**

   Необходимо создать массив потоков, через который и будет осуществляться управление. Если необходимо знать идентификатор выполняющегося потока, то используется `Thread.CurrentThread.ManagedThreadId`. Непосредственно в поток передается функция, удовлетворяющая сигнатуре делегата.

Выводы

В данной лабораторной работе реализован один из способ создания параллельных задач - через объект потока `Thread`. Были на примере продемонстрированы способ управления потоками через массив потоков.